

```
manchodu@Srini:~/OSLAB$ nano m3.c
manchodu@Srini:~/OSLAB$ gcc -o m3 m3.c
manchodu@Srini:~/OSLAB$ ./m3
Usage: ./m3 <source> <low_level_dest> <standard_dest>
manchodu@Srini:~/OSLAB$ |
```

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <time.h>

#define BUFFER_SIZE 4096

void low_level_copy(const char *source, const char *destination)
{
    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    src = open(source, O_RDONLY);

    if (src == -1)
    {
        perror("open source");
        exit(1);
    }

    dest = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest == -1)
    {
        perror("open destination");
        close(src);
        exit(1);
    }

    while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0)
    {
        bytes_written = write(dest, buffer, bytes_read);

        if (bytes_written != bytes_read)
        {
            perror("write");
            close(src);
            close(dest);
            exit(1);
        }
    }

    if (bytes_read == -1)
        perror("read");

    /* Demonstrate lseek() */
}
```

```

        perror("read");

    /* Demonstrate lseek() */
    lseek(src, 0, SEEK_SET);

    close(src);
    close(dest);
}

void standard_library_copy(const char *source, const char *destination)
{
    FILE *src, *dest;
    char buffer[BUFFER_SIZE];
    size_t bytes_read;

    src = fopen(source, "rb");

    if (src == NULL)
    {
        perror("fopen source");
        exit(1);
    }

    dest = fopen(destination, "wb");

    if (dest == NULL)
    {
        perror("fopen destination");
        fclose(src);
        exit(1);
    }

    while ((bytes_read = fread(buffer, 1, BUFFER_SIZE, src)) > 0)
    {
        fwrite(buffer, 1, bytes_read, dest);
    }

    fclose(src);
    fclose(dest);
}

int main(int argc, char *argv[])
{
    clock_t start, end;
    double low_level_time, standard_time;

    if (argc != 4)
    {
        printf("Usage: %s <source> <low_level_dest> <standard_dest>\n", argv[0]);
        return 1;
    }
}

```

```

manchodu@Srini:~/OSLAB$ nano m4.c
manchodu@Srini:~/OSLAB$ gcc -o m4 m4.c
manchodu@Srini:~/OSLAB$ ./m4
=====
I/O REDIRECTION USING dup2()
=====
open input.txt: No such file or directory
manchodu@Srini:~/OSLAB$ |

```

```
manchodu@Srini:~/OSLAB$ cat m4.c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int input_fd;
    int output_fd;

    printf("=====\n");
    printf("      I/O REDIRECTION USING dup2()\n");
    printf("=====\n");

    /*
     * Open input file.
     * This file will become standard input.
     */
    input_fd = open("input.txt", O_RDONLY);

    if (input_fd == -1)
    {
        perror("open input.txt");
        return 1;
    }

    /*
     * Open output file.
     * This file will become standard output.
     */
    output_fd = open(
        "output.txt",
        O_WRONLY | O_CREAT | O_TRUNC,
        0644
    );

    if (output_fd == -1)
    {
        perror("open output.txt");
        close(input_fd);
        return 1;
    }
}
```

```

/* Open output file.
 * This file will become standard output.
 */
output_fd = open(
    "output.txt",
    O_WRONLY | O_CREAT | O_TRUNC,
    0644
);

if (output_fd == -1)
{
    perror("open output.txt");
    close(input_fd);
    return 1;
}

/*
 * Redirect standard input (file descriptor 0)
 * to input.txt.
 */
if (dup2(input_fd, STDIN_FILENO) == -1)
{
    perror("dup2 stdin");
    close(input_fd);
    close(output_fd);
    return 1;
}

/*
 * Redirect standard output (file descriptor 1)
 * to output.txt.
 */
if (dup2(output_fd, STDOUT_FILENO) == -1)
{
    perror("dup2 stdout");
    close(input_fd);
    close(output_fd);
    return 1;
}

/*
 * Original descriptors are no longer needed.
 */
close(input_fd);
close(output_fd);

/*
 * From this point:
 *
 * stdin -> input.txt

```

```

{
    perror("dup2 stdin");
    close(input_fd);
    close(output_fd);
    return 1;
}

/*
 * Redirect standard output (file descriptor 1)
 * to output.txt.
 */
if (dup2(output_fd, STDOUT_FILENO) == -1)
{
    perror("dup2 stdout");
    close(input_fd);
    close(output_fd);
    return 1;
}

/*
 * Original descriptors are no longer needed.
 */
close(input_fd);
close(output_fd);

/*
 * From this point:
 *
 * stdin  -> input.txt
 * stdout -> output.txt
 */

char buffer[1024];
ssize_t bytes_read;

printf("This output is redirected to output.txt\n");

while ((bytes_read = read(STDIN_FILENO,
                        buffer,
                        sizeof(buffer))) > 0)
{
    write(STDOUT_FILENO, buffer, bytes_read);
}

return 0;
}

```